PROJECTS TASKS

Hex's Software

SUBMITTED BY MUQADDAS FATIMA

❖ PROJECT 1 – FIBONACCI GENERATOR

Introduction

The Fibonacci Series is a sequence of numbers where each number is the sum of the two previous numbers.

The sequence starts with:

0, 1, 1, 2, 3, 5, 8, 13, 21, 34, ...

Mathematical Formula:

$$F(n) = F(n-1) + F(n-2)$$

Objective

To create a Python program that:

- Takes input from the user
- Generates Fibonacci numbers
- Displays the series up to the given number of terms

Concepts Used

- Functions
- Loops (for loop)
- Variables
- User input
- Basic mathematical logic

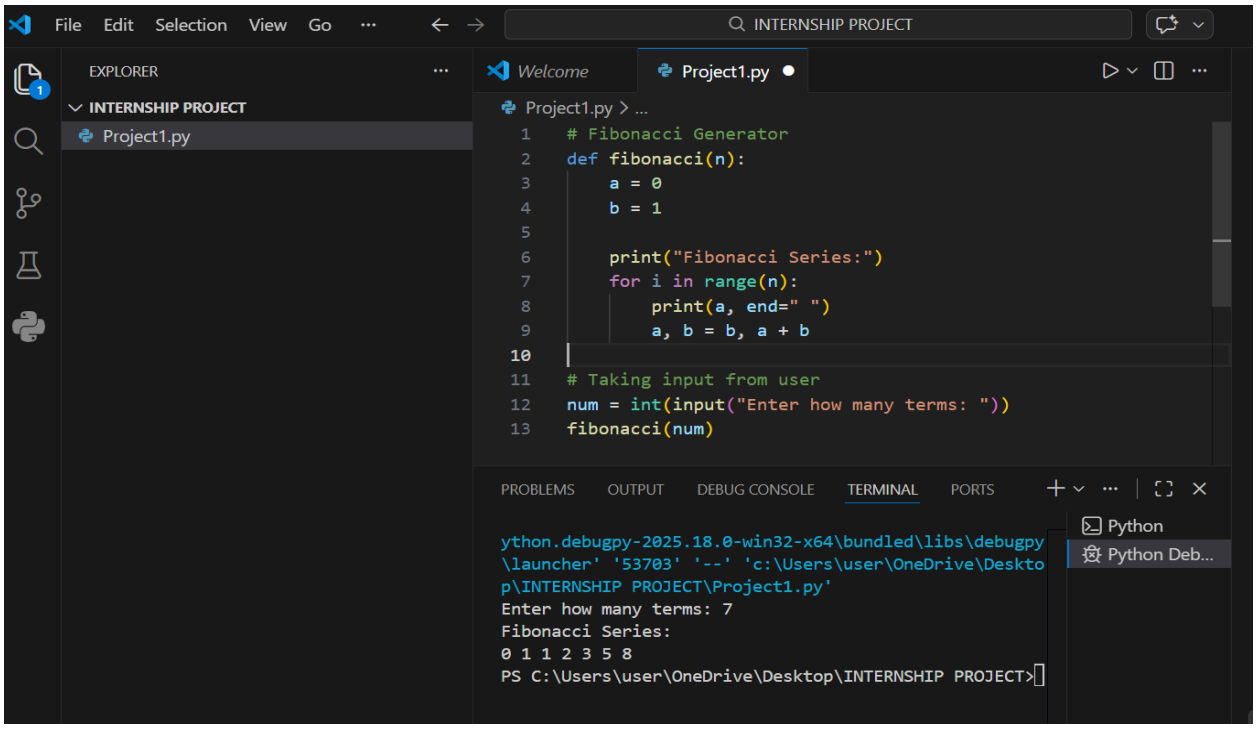
Working Explanation

1. The program asks the user how many terms they want.
2. It starts with two variables:
   - a = 0
   - b = 1
3. It prints the first number.
4. Then updates values:
   - New value = a + b
   - Shift numbers forward
5. Loop continues until required terms are printed.

Applications

- Used in mathematics
- Used in algorithms
- Used in financial market analysis
- Used in nature modeling (spirals, shells)

CODE IN VISUAL STUDIO WITH OUTPUT:



❖ PROJECT 2 – VOICE ASSISTANT

Introduction

A Voice Assistant is a program that:

- Listens to voice commands
- Converts speech to text
- Processes the command
- Performs tasks

- Responds using voice output

Examples in real life:

- Siri
- Alexa
- Google Assistant

Objective

To build a basic voice assistant in Python that can:

- Tell time
- Open websites
- Search Google
- Respond using speech
- Exit on command

Libraries Used

1. **speech_recognition**
   → Converts voice to text
2. **pyttsx3**
   → Converts text to speech
3. **datetime**
   → Gets current time
4. **webbrowser**
   → Opens websites
5. **pyaudio**
   → Enables microphone access

Working Explanation

Step 1:

Microphone listens using speech_recognition.

Step 2:

Voice is converted into text.

Step 3:

Program checks keywords like:

- "time"

- "open google"
- "exit"

Step 4:

If condition matches:

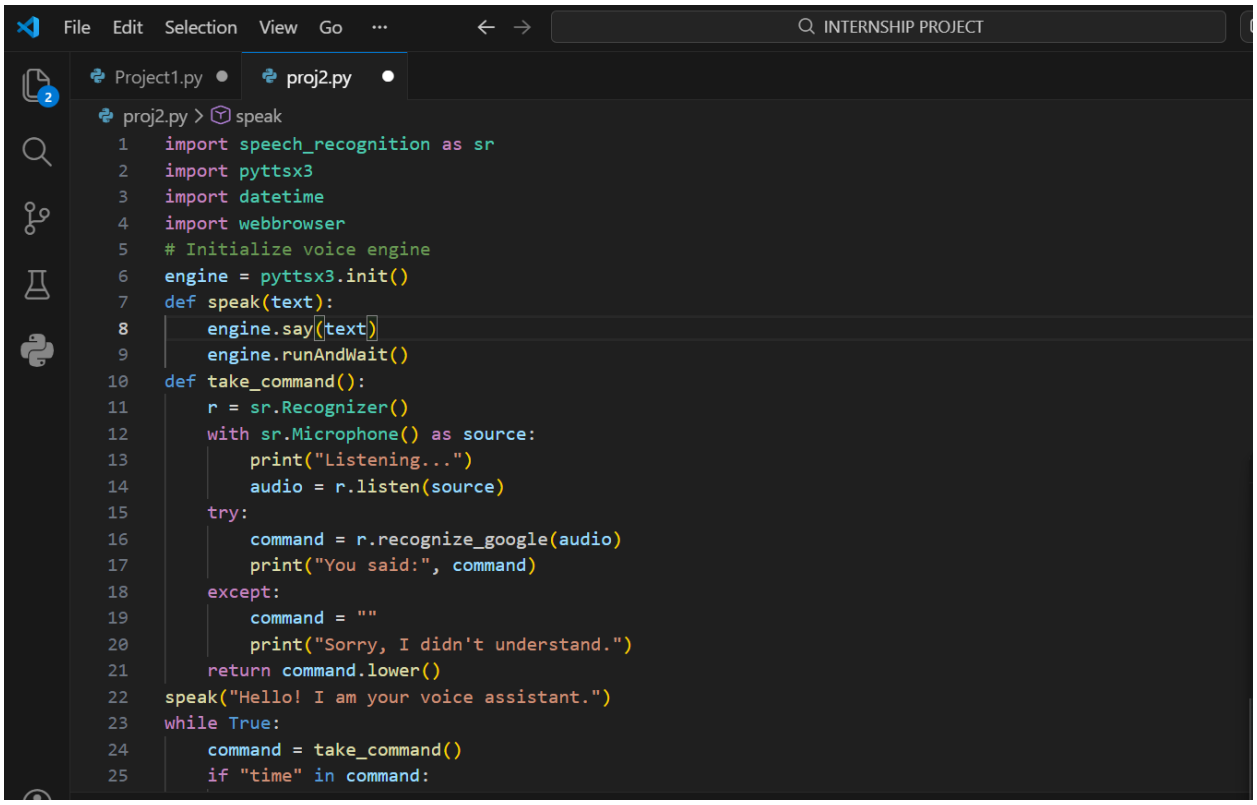
- It performs action
- Speaks response using pyttsx3

The program runs in a loop until the user says ***"exit***".

Applications

- Home automation
- Task automation
- AI assistants
- Smart devices
- Accessibility tools

CODE IN VISUAL:




```python
import speech_recognition as sr
import pyttsx3
import datetime
import webbrowser
# Initialize voice engine
engine = pyttsx3.init()
def speak(text):
    engine.say(text)
    engine.runAndWait()
def take_command():
    r = sr.Recognizer()
    with sr.Microphone() as source:
        print("Listening...")
        audio = r.listen(source)
    try:
        command = r.recognize_google(audio)
        print("You said:", command)
    except:
        command = ""
        print("Sorry, I didn't understand.")
    return command.lower()
speak("Hello! I am your voice assistant.")
while True:
    command = take_command()
    if "time" in command:
```

```python
def take_command():
    except:
        command = ""
        print("Sorry, I didn't understand.")
    return command.lower()
speak("Hello! I am your voice assistant.")
while True:
    command = take_command()
    if "time" in command:
        time = datetime.datetime.now().strftime("%H:%M:%S")
        speak("The time is " + time)
    elif "open google" in command:
        webbrowser.open("https://google.com")
        speak("Opening Google")
    elif "exit" in command:
        speak("Goodbye!")
        break
```

```
Listening...
Sorry, I didn't understand.
Listening...
You said: my name
Listening...
You said: your name
Listening...
You said: what is time
Listening...
You said: which hobby do you like
Listening...
You said: what are you doing in free
Listening...
You said: I am very happy today
Listening...
You said: great
Listening...
You said: you can working voice assistant
Listening...
You said: thank
Listening...
You said: bye-bye
Listening...
```

PROJECT 3 – TIC-TAC-TOE GAME

Introduction

Tic-Tac-Toe is a 2-player game played on a 3x3 grid.

In this project:

- User plays as "X"
- Computer plays as "O"
- Computer move is generated randomly

Objective

To create a Python game where:

- User plays against computer
- Game checks for winner
- Declares win, lose, or draw

Concepts Used

- Lists (to represent board)
- Loops
- Conditional statements
- Functions
- Random module
- Game logic

Working Explanation

1. A list of 9 elements represents board.
2. **Board positions:**
   0 1 2
   3 4 5
   6 7 8
3. User enters position number.
4. **Program checks:**
   - Is position empty?
   - If yes → place "X"
5. Computer randomly chooses empty spot.
6. After every move:
   - Program checks winning conditions.
7. **Game ends if:**

- User wins
- Computer wins
- Board is full (Draw)

Applications

- Game development basics
- Logic building
- Understanding AI behavior
- Strengthening programming skill

CODE IN VISUAL WITH OUTPUT:

proj3.py

```python
import random

board = [" " for x in range(9)]

def print_board():
    print(board[0] + "|" + board[1] + "|" + board[2])
    print("-+-+-")
    print(board[3] + "|" + board[4] + "|" + board[5])
    print("-+-+-")
    print(board[6] + "|" + board[7] + "|" + board[8])

def check_winner(player):
    win_conditions = [
        [0,1,2], [3,4,5], [6,7,8],
        [0,3,6], [1,4,7], [2,5,8],
        [0,4,8], [2,4,6]
    ]

    for condition in win_conditions:
        if board[condition[0]] == board[condition[1]] == board[condition[2]] == player:
            return True
    return False

def computer_move():
    available = [i for i in range(9) if board[i] == " "]
    move = random.choice(available)
    board[move] = "O"
```

(variable) condition: list[int]

proj3.py

```python

while True:
    print_board()
    move = int(input("Enter your move (0-8): "))

    if board[move] == " ":
        board[move] = "X"
    else:
        print("Invalid move!")
        continue

    if check_winner("X"):
        print_board()
        print("You win!")
        break

    if " " not in board:
        print("It's a draw!")
        break

    computer_move()

    if check_winner("O"):
        print_board()
        print("Computer wins!")
        break
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Python

```
Enter your move (0-8): 4
 | |
-+-+-
 |X|
-+-+-
 |O|
Enter your move (0-8): 8
 | |
-+-+-
O|X|
-+-+-
 |O|X
Enter your move (0-8): 2
 | |X
-+-+-
O|X|
-+-+-
O|O|X
Enter your move (0-8):
```